

DML (Data Manipulation Language)

Activity 01 - Sample

Imagine we have a table named **Attendance** to store data for the QR attendance system. Let's assume this table has the following 3 columns:

- student_id (Number)
- name (Text)
- status (Text - either 'Present' or 'Absent')

Here are your 3 tasks. Try writing the corresponding SQL queries for each:

Task 1: (INSERT) We need to add a new record to the table for a student who just scanned their QR code.

Task 2: (UPDATE) Let's say a mistake happened, and Kamal's status was accidentally recorded as 'Absent' in the database. We need to update his status back to 'Present'. (Use Kamal's student_id of 105 to target the exact row).

UPDATE attendance

SET status = 'Present'

WHERE student id = 105;

Task 3: (DELETE) We need to remove an old dummy record that was added just to test the system earlier. The student_id for this test record is 105.

[illegible][illegible]

Activity 1: E-Commerce Database Operations

Scenario: You are tasked with managing the database for an electronics e-commerce store. The database is structured with three tables: Customers, Products, and Orders.

Initial Database State:

Table 1: Customers

customer_id	name	email
1	Alice Smith	alice@email.com
2	Bob Jones	bob@email.com

Table 2: Products

product_id	product_name	price	stock_quantity
101	Laptop	1200.00	10
102	Wireless Mouse	25.00	50
103	Keyboard	45.00	30

Table 3: Orders

order_id	customer_id	product_id	quantity	status
1001	1	101	1	Shipped
1002	2	102	2	Processing

SQL Tasks (Write the necessary SQL statements):

Data Insertion (INSERT)

- New Customer Registration:** Add a new customer, 'Charlie Brown' (ID: 3, Email: 'charlie@email.com'), to the Customers table.
- New Product Listing:** Introduce a new product, 'Webcam' (ID: 104, Price: 60.00, Stock: 20), to the Products table.
- New Order Placement:** Record a new order (ID: 1003) from Charlie Brown (customer ID 3) for 1 Webcam (product ID 104), with the initial status 'Processing'.

```
//INSERT INTO Orders VALUES(1003,3,101,1,'Processing');
```

Data Modification (UPDATE)

1. **Update Order Status:** Change the status of Charlie's order (order ID 1003) from 'Processing' to 'Shipped'.

```
//UPDATE Orders

SET status = 'Shipped'

-WHERE order_id = 1003;
```

2. **Update Product Stock:** Decrease the stock_quantity for the 'Webcam' (product ID 104) by 1 (new stock: 19) to reflect the sale to Charlie.

```
UPDATE Products

SET stock_quantity = 19

WHERE product_id = 104;
```

Data Deletion (DELETE)

1. **Cancel Order:** Remove order ID 1002, which was canceled by Bob Jones, from the Orders table.

```
DELETE FROM Orders

WHERE order_id = 1002;
```

=====

=====

=====

Activity 2: Local Library System

The system uses three main tables: **Members**, **Books**, and **Borrowings**.

Table 1: Members

member_id	full_name	membership_type
1	John Doe	Regular
2	Jane Smith	Premium

Table 2: Books

book_id	title	author	available_copies
---------	-------	--------	------------------

In SQL, you can update or delete multiple rows at the exact same time. This is done by using specific conditions in the **WHERE** clause.

There are two common methods to achieve this:

Method 1: Using the IN Operator

Use the **IN** keyword when you know the exact IDs or specific values of the rows you want to modify.

1. UPDATE Example: Changing the status of specific orders (Order IDs 101, 102, and 103) to 'Shipped'.

```
SQL
UPDATE Orders
SET status = 'Shipped'
WHERE order_id IN (101, 102, 103);
```

2. DELETE Example: Removing specific users (User IDs 5, 8, and 12) from the system.

```
SQL
DELETE FROM Users
WHERE user_id IN (5, 8, 12);
```

Method 2: Using a Common Condition

Use this method when you want to modify all rows that share a certain characteristic or condition (using operators like **=**, **>**, **<**, **!=**).

1. UPDATE Example: Changing the status to 'Shipped' for ALL orders that are currently marked as 'Processing'.

```
SQL
UPDATE Orders
SET status = 'Shipped'
WHERE status = 'Processing';
```

2. DELETE Example: Removing ALL orders from the table that have a 'Cancelled' status.

```
SQL
DELETE FROM Orders
WHERE status = 'Cancelled';
```

CRITICAL WARNING: Always double-check your **WHERE** clause before executing an UPDATE or DELETE query. If you forget to include the **WHERE** clause (for example: **DELETE FROM Users;**), you will accidentally modify or delete **EVERY SINGLE ROW** in that table!

Activity: Multiplayer Game Server Maintenance

Context: You are the database administrator for a popular online multiplayer open-world game. Your current task involves performing essential bulk updates and server cleanups across the `Players` and `Player_Vehicles` tables.

Current Database Snapshot:

Table 1: Players

player_id	username	account_status
1	Speedster99	Active
2	RuleBreaker	Banned
3	PixelNinja	Active
4	TrollFace	Banned
5	AFK_King	Inactive

Table 2: Player_Vehicles

vehicle_id	owner_id	vehicle_type	vehicle_condition
101	1	Sports Car	Damaged
102	1	Motorcycle	Perfect
103	2	Armored Truck	Damaged
104	3	Motorcycle	Damaged
105	4	Helicopter	Perfect

Required SQL Operations (Write the Queries):

- Grant VIP Status (UPDATE with IN):** Award 'VIP' account status to players with `player_ids` 1, 3, and 5, as designated by the game developers for a special reward.
- Repair Motorcycles (Conditional UPDATE):** Execute a server-wide event that automatically sets the `vehicle_condition` to 'Perfect' for all vehicles of the `vehicle_type` 'Motorcycle' in the `Player_Vehicles` table.
- Remove Glitched Vehicles (DELETE with IN):** Permanently delete vehicles with `vehicle_ids` 103 and 105 from the `Player_Vehicles` table due to a system glitch.
- Cleanup Banned Accounts (Conditional DELETE):** Perform the monthly server cleanup by

deleting all entries from the Players table where the account_status is 'Banned'.

=====

=====

Activity 02: Employee Database Operations

Objective: You are tasked with managing the Human Resources employee database for a company. The database contains one table, Employees, which stores staff information including department, salary, and a recent performance score (out of 100). You need to write SQL queries to perform various bulk data updates and cleanups based on defined criteria.

Database Snapshot:

Table: Employees				
emp_id	name	department	salary	performance_score
1	Alice	Sales	45000	65
2	Bob	IT	60000	85
3	Charlie	Sales	38000	40
4	Diana	HR	55000	92
5	Evan	IT	42000	50
6	Fiona	Marketing	48000	75

Required SQL Queries (Use ONLY the following operators: =, <, >, <=, >=):

- Salary Adjustment (UPDATE):** Apply a standard company salary increase to the 'Sales' department. Set the salary to 50000 for all employees in the 'Sales' department.
- Training Program Assignment (UPDATE):** Employees with a performance_score strictly less than 50 must be reassigned to a new training program. Change the department to 'Training' for all employees with a performance_score < 50.
- Top Performer Raise (UPDATE):** High-achieving employees are receiving a significant pay increase. Set the salary to 75000 for all employees whose performance_score is 85 or greater (>= 85).
- Outsourcing Cleanup (DELETE):** The company is outsourcing all Marketing roles. Remove all employees from the table where the department is 'Marketing'.
- Role Removal Policy (DELETE):** Implement a new policy to eliminate roles with low compensation. Delete all employees whose salary is strictly below 40000 (< 40000).

